Платформено-независими програмни езици

Лекция 2

Класове и обекти

Дефиниране на класове, капсулиране, конструктори, статични членове и записи (record)

2

проф. д-р Даниела Гоцева
Учебна 2026/2027 г.

Съдържание на лекцията

1. Класове и обекти – основни понятия
2. Полета, методи и капсулиране
3. Конструктори и инициализация
4. Методи: параметри и резултат
5. Статични членове и константи
6. Записи (record)

01

Класове и обекти – основни понятия

Клас като нов тип, обект като инстанция

Какво е клас?

- **Клас** – съвкупност от данни (полета) и методи за тяхната обработка
- Класът е **шаблон** (чертеж), по който се създават обекти
- С всеки нов клас **разширяваме системата от типове** на езика
- Класът описва:
  - **състояние** – какви данни пази всеки обект
  - **поведение** – какви операции може да извършва
- Изходният файл носи името на публичния клас: `Vect2D.java`

Клас и обект

Клас

- описание, шаблон
- един в програмата
- съществува по време на компилиране
- пример: `Student`

Обект (инстанция)

- конкретен екземпляр на класа
- може да има много обекти
- създава се с `new` по време на изпълнение
- пример: студентът Иван, ф.н. 12345

Първи клас: Vect2D

- Двумерен вектор с координати `x` и `y`
- Полетата са `private` – скрити отвън
- Конструкторът задава началното състояние
- Методите работят с данните на обекта

```
public class Vect2D {
    private double x;
    private double y;

    public Vect2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double length() {
        return Math.sqrt(x * x + y * y);
    }

    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}
```

Създаване и използване на обекти

- `new` заделя памет и извиква конструктор
- Променливата пази **референция** към обекта
- Достъп до членове: `обект.метод(аргументи)`
- `println` извиква автоматично `toString()`

```
Vect2D a = new Vect2D(3, 4);
Vect2D b = new Vect2D(1, 0);

System.out.println(a.length());   // 5.0
System.out.println(a);            // (3.0, 4.0)

Vect2D c = a;           // втора референция
Vect2D d = null;        // няма обект
// d.length();  -> NullPointerException
```

Обекти в паметта

- Локалните променливи живеят в **стека** (stack)
- Обектите се създават в **хийпа** (heap)
- Променлива от референтен тип съдържа **адрес** на обект
- Две променливи може да сочат към **един и същ** обект
- Когато никоя референция не сочи обект, той става „боклук“
- **Garbage collector** освобождава паметта автоматично – няма `delete`

Типични видове класове

Вид	Предназначение	Примери
Бизнес обекти (данни)	представят обекти от предметната област, често от БД	`Employee`, `Order`, `Invoice`
Алгебрични	математически типове	`Vect2D`, `Complex`, `Fraction`
Контейнери	съхраняват колекции от обекти	`ArrayList`, `HashMap`
Вход/изход и мрежа	достъп до външни данни	`Files`, `Socket`, `HttpClient`
Услуги (services)	бизнес логика, алгоритми	`PaymentService`, `Validator`
Помощни (utility)	само статични методи	`Math`, `Arrays`, `Objects`

Класове в Python и в Java

Python

- `class Student:`
- полетата се създават в `__init__`
- `self` – явен първи параметър
- няма истински `private` – само конвенция `_name`
- `__str__` за текстово представяне

Java

- `public class Student { ... }`
- полетата се **декларират** с тип
- `this` – неявен, пише се при нужда
- `private` се проверява от компилатора
- `toString()` за текстово представяне

UML клас диаграма

- Стандартна графична нотация за описание на класове
- Правоъгълник с три части:
  - **име** на класа
  - **полета**: `- balance: double`
  - **методи**: `+ deposit(amount: double): void`
- Видимост: `+` public, `-` private, `#` protected, `~` package
- Статичните членове са подчертани
- Инструменти: draw.io, PlantUML, Mermaid, плъгини за Eclipse

02

Полета, методи и капсулиране

Модификатори за достъп, get/set методи, this

Модификатори за достъп

Модификатор	Клас	Пакет	Наследник	Навсякъде
`public`	да	да	да	да
`protected`	да	да	да	не
без модификатор (package-private)	да	да	не	не
`private`	да	не	не	не

Нива на контрол на достъпа

- **На ниво клас (top level):** `public` или package-private
- **На ниво член (поле, метод, конструктор):** `public`, `protected`, package-private, `private`
- Правило: давайте **най-ограничения** достъп, който работи
- Полетата – почти винаги `private`
- Публичните методи образуват **интерфейса** на класа към света
- Помощните методи – `private`



Капсулиране: обектът сам пази своите данни

Външният код не променя полетата директно, а използва методи. Така класът гарантира, че състоянието му е винаги коректно (инвариант).

Капсулиране: банкова сметка

```
public class BankAccount {
    private final String iban;
    private double balance;      // никой отвън не го променя директно

    public BankAccount(String iban) {
        this.iban = iban;
    }

    public double getBalance() { return balance; }
    public String getIban()    { return iban; }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Сумата трябва да е > 0");
        }
        balance += amount;
    }
}
```

Капсулиране: теглене и проверки

- Методът решава **дали** операцията е допустима
- Невалидна операция → `false` или изключение
- Външният код не може да „развали“ сметката

```
public boolean withdraw(double amount) {
    if (amount <= 0 || amount > balance) {
        return false;
    }
    balance -= amount;
    return true;
}

BankAccount acc = new BankAccount("BG80BNBG");
acc.deposit(100);
acc.withdraw(30);                  // true
acc.withdraw(500);                 // false
// acc.balance = 1_000_000;        // грешка!
System.out.println(acc.getBalance()); // 70.0
```

Методи за достъп (get / set)

- **Getter** – само чете: `getName()`, за `boolean` – `isPassed()`
- **Setter** – променя и може да **проверява** входа
- Не правете сетър за всяко поле „по навик“!

```java
public class Student {
    private String name;
    private int grade;

    public String getName() { return name; }

    public void setGrade(int grade) {
        if (grade < 2 || grade > 6) {
            throw new IllegalArgumentException(
                "Невалидна оценка: " + grade);
        }
        this.grade = grade;
    }

    public boolean isPassed() {
        return grade >= 3;
    }
}
```

Ключовата дума this

- `this` – референция към **текущия обект**
- Нужна е, когато параметър **скрива** поле със същото име
- Може да се подаде като аргумент или да се върне
- Връщането на `this` позволява **верижни извиквания**

```java
public class Counter {
    private int value;

    public Counter add(int value) {
        this.value += value;   // поле += параметър
        return this;           // за верижно извикване
    }

    public int getValue() {
        return value;
    }
}

int r = new Counter().add(5).add(10).getValue();
// r == 15
```

Методите toString, equals и hashCode

- Наследяват се от `Object` за всеки клас
- `toString()` – текстово представяне
- `equals()` – логическо равенство
- `hashCode()` – задължително заедно с `equals`
- IDE ги генерира: Eclipse → Source → Generate…

```java
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Vect2D other)) return false;
    return Double.compare(x, other.x) == 0
        && Double.compare(y, other.y) == 0;
}

@Override
public int hashCode() {
    return Objects.hash(x, y);
}
```

Пример: клас Time с проверки

```
public final class Time {
    private final int minutes;   // от 00:00
    public Time(int h, int m) {
        if (h < 0 || h > 23 || m < 0 || m > 59)
            throw new IllegalArgumentException(
                    h + ":" + m);
        minutes = h * 60 + m;
    }
    public Time plus(int mins) {
        int t = Math.floorMod(minutes + mins,
                              24 * 60);
        return new Time(t / 60, t % 60);
    }

    @Override public String toString() {
        return "%02d:%02d".formatted(
                minutes / 60, minutes % 60);
    }
}
```

03

Конструктори и инициализация

Конструктор по подразбиране, претоварване, блокове за инициализация

Конструктори

- Специален метод, който се извиква при `new`
- Името съвпада с името на класа
- **Няма тип на връщане** – дори `void`
- Всеки клас има поне един конструктор
- Ако не декларираме нито един, компилаторът създава **конструктор по подразбиране** – без параметри
- Ако декларираме конструктор с параметри, конструктор по подразбиране **не се създава**

Претоварени конструктори и this(...)

- Няколко конструктора с различни параметри
- `this(...)` извиква друг конструктор на същия клас
- Трябва да е **първият оператор**
- Така логиката е на едно място

```
public class Vect2D {
    private final double x, y;

    public Vect2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public Vect2D() {
        this(0, 0);              // нулев вектор
    }

    public Vect2D(Vect2D other) {
        this(other.x, other.y); // копиращ
    }
}
```

Стойности по подразбиране на полетата

Тип на полето	Стойност по подразбиране
`byte`, `short`, `int`, `long`	`0`
`float`, `double`	`0.0`
`char`	`'\u0000'`
`boolean`	`false`
референтен тип (`String`, масив, обект)	`null`

Ред на инициализация на обекта

- 1. Стойности по подразбиране
- 2. Инициализатори на полета и блокове `{ }` – в реда на кода
- 3. Тялото на конструктора

```
public class Demo {
    private int a = 10;           // (2)
    private int b;

    {                             // блок (2)
        b = a * 2;
        System.out.println("b=" + b);
    }

    public Demo() {               // (3)
        a = 100;
        System.out.println("a=" + a);
    }
}
```

Статични блокове за инициализация

- Изпълняват се **веднъж**, при зареждане на класа
- Служат за инициализация на **статични** полета
- Може да има няколко – изпълняват се в реда на кода
- Алтернатива: `private static` метод

```
public class Config {
    static final Map<String, String> DEFAULTS;

    static {
        DEFAULTS = new HashMap<>();
        DEFAULTS.put("lang", "bg");
        DEFAULTS.put("theme", "light");
    }

    // алтернатива – може да се преизползва
    private static int initPort() {
        return 8080;
    }
    static final int PORT = initPort();
}
```

Полета final и неизменяеми обекти

- `final` поле получава стойност **точно веднъж** – в декларацията или в конструктора
- **Неизменяем** (immutable) обект – състоянието му не се променя след създаване
- Предимства: безопасност, лесно тестване, работа с нишки
- „Промяна“ = нов обект

```
public final class Vect2D {
    private final double x;
    private final double y;

    public Vect2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public Vect2D plus(Vect2D o) {
        return new Vect2D(x + o.x, y + o.y);
    }

    public Vect2D scale(double k) {
        return new Vect2D(k * x, k * y);
    }
}
```

Жизнен цикъл на обекта

- **Създаване** – `new`: заделяне на памет, стойности по подразбиране, конструктор
- **Използване** – чрез една или повече референции
- **Недостижимост** – никоя референция не сочи обекта
- **Освобождаване** – garbage collector освобождава паметта, когато прецени
- Няма деструктори: ресурси (файлове, връзки) се затварят **явно** – `close()`, try-with-resources (лекция 7)



Предпочитайте неизменяеми класове, освен ако няма причина за обратното

Принцип от **Effective Java** (J. Bloch). `String`, `Integer`, `LocalDate` и записите (`record`) са неизменяеми.

04

Методи: параметри и резултат

Предаване по стойност, varargs, статична и обектна форма

Предаване на примитивни параметри

- В Java **всички** параметри се предават **по стойност**
- Методът получава **копие** на стойността
- Промяната на параметъра не засяга оригинала

```
static void increment(int n) {
    n++;
    System.out.println("вътре: " + n);  // 6
}

public static void main(String[] args) {
    int x = 5;
    increment(x);
    System.out.println("отвън: " + x);  // 5
}
```

Предаване на референции

- Копира се **референцията**, не обектът
- Методът може да промени **полетата** на обекта
- Но не може да накара оригиналната променлива да сочи друг обект

```
static void reset(BankAccount acc) {
    acc.withdraw(acc.getBalance());  // променя обекта
}

static void replace(BankAccount acc) {
    acc = new BankAccount("BG00");   // само локално!
}

BankAccount a = new BankAccount("BG11");
a.deposit(100);
reset(a);
System.out.println(a.getBalance());  // 0.0
replace(a);
System.out.println(a.getIban());     // BG11
```

Променлив брой параметри (varargs)

- Когато броят на аргументите не е известен предварително
- Синтаксис: `Тип... име`
- Вътре в метода параметърът е **масив**
- Само един и винаги **последен** параметър

```
static double average(double... values) {
    if (values.length == 0) return 0;
    double sum = 0;
    for (double v : values) {
        sum += v;
    }
    return sum / values.length;
}

average();                      // 0.0
average(5);                     // 5.0
average(4, 5, 6);               // 5.0
average(new double[]{3, 6});// 4.5

String s = String.format("%s-%d", "A", 1);
```

Връщана стойност

- Методът връща примитивна стойност или референция
- `return` прекратява метода
- **Ковариантен тип на връщане** – може да се върне и наследник на декларирания тип
- Неизменяемите методи връщат нов обект

```
public Vect2D normalize() {
    double len = length();
    if (len == 0) {
        return this;
    }
    return new Vect2D(x / len, y / len);
}

public double dot(Vect2D o) {
    return x * o.x + y * o.y;
}
```

Пример: клас Fraction (дроб)

```
public final class Fraction {
    private final int num;      // числител
    private final int den;      // знаменател
    public Fraction(int num, int den) {
        if (den == 0) throw new ArithmeticException("Знаменател 0");
        int g = gcd(Math.abs(num), Math.abs(den));
        int sign = den < 0 ? -1 : 1;
        this.num = sign * num / g;
        this.den = sign * den / g;
    }

    private static int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }

    public Fraction plus(Fraction o) {
        return new Fraction(num * o.den + o.num * den, den * o.den);
    }

    @Override public String toString() { return num + "/" + den; }
}
```

Използване на Fraction

- Конструкторът **нормализира** дробта – винаги несъкратима, знакът в числителя
- Помощният метод `gcd` е `private static` – детайл от реализацията
- Обектите са неизменяеми – `plus` връща нова дроб

```
Fraction a = new Fraction(1, 2);
Fraction b = new Fraction(2, -6);

System.out.println(b);           // -1/3
System.out.println(a.plus(b));   // 1/6

Fraction c = new Fraction(4, 8);
System.out.println(c);           // 1/2

// new Fraction(1, 0);
//   -> ArithmeticException
```

Статична или обектна форма на операция?

```
public final class Vect2D {
    // ... полета и конструктор

    // Обектна форма: a.plus(b)
    public Vect2D plus(Vect2D o) {
        return new Vect2D(x + o.x, y + o.y);
    }

    // Статична форма: Vect2D.sum(a, b)
    public static Vect2D sum(Vect2D a, Vect2D b) {
        return new Vect2D(a.x + b.x, a.y + b.y);
    }
}

Vect2D r1 = a.plus(b);
Vect2D r2 = Vect2D.sum(a, b);
```

Статичната форма подчертава симетрията; обектната е по-гъвкава – може да се припокрие в наследник

Композиция: обекти като полета

- Клас може да съдържа обекти от други класове
- Връзка „**има**“ (has-a): курсът **има** студенти
- Сложни обекти се изграждат от прости
- Основен начин за преизползване на код

```
public class Course {
    private final String title;
    private final List<Student> students =
            new ArrayList<>();

    public Course(String title) {
        this.title = title;
    }

    public void enroll(Student s) {
        students.add(s);
    }

    public int size() {
        return students.size();
    }
}
```

Масиви от обекти

- Масивът съдържа **референции**, не обекти
- След `new Student[3]` всички елементи са `null`
- Всеки обект се създава отделно

```
Student[] group = new Student[3];
System.out.println(group[0]);   // null

group[0] = new Student("Ана", 101, 6);
group[1] = new Student("Боян", 102, 4);
group[2] = new Student("Вера", 103, 5);

double sum = 0;
for (Student s : group) {
    sum += s.grade();
}
System.out.printf("Среден: %.2f%n",
        sum / group.length);
```

Защитни копия

- Ако полето е **изменяем** обект, гетърът може да „изпусне“ вътрешното състояние
- Решение: връщаме копие или неизменяем изглед
- Същото – при получаване в конструктора

```
public class Course {
    private final List<Student> students;

    public Course(List<Student> list) {
        this.students = new ArrayList<>(list);
    }

    public List<Student> getStudents() {
        return List.copyOf(students);
    }
}
// course.getStudents().clear();
//   -> UnsupportedOperationException
```

05

Статични членове и константи

Членове на класа, а не на обекта

Статични полета

- `static` поле е **общо за всички обекти** на класа
- Памет за него се заделя еднократно
- Достъпва се през името на класа – без обект
- Пример: брояч на създадените обекти

```
public class Student {
    private static int count = 0;   // общо
    private final int id;           // за всеки обект
    private final String name;

    public Student(String name) {
        this.name = name;
        this.id = ++count;          // уникален номер
    }

    public static int getCount() {
        return count;
    }
}

new Student("Ана");
new Student("Боян");
System.out.println(Student.getCount());  // 2
```

Статични методи

- Извикват се през класа: `Math.max(a, b)`
- **Нямат** `this`
- Подходящи за помощни функции
- **Фабричен метод** – конструктор с описателно име

```
public final class Vect2D {
    private final double x, y;

    private Vect2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public static Vect2D of(double x, double y) {
        return new Vect2D(x, y);
    }

    public static Vect2D polar(double r, double a) {
        return new Vect2D(r * Math.cos(a),
                          r * Math.sin(a));
    }
}
```

Какво достъпват методите?

Метод	Полета на обекта	Методи на обекта	Статични членове	`this`
обектен (instance)	да, директно	да, директно	да, директно	да
статичен (static)	само чрез референция	само чрез референция	да, директно	не

Константи

- `static final` – константа на класа
- Име с ГЛАВНИ_БУКВИ
- Стойността се задава веднъж
- За фиксиран набор от стойности – предпочитайте `enum`

```java
public class Physics {
    public static final double G = 9.80665;
    public static final int MAX_SPEED = 300_000;

    private Physics() { }   // не се инстанцира

    public static double fallTime(double h) {
        return Math.sqrt(2 * h / G);
    }
}

double t = Physics.fallTime(20);
```

Статичен импорт

- `import static` – статичните членове без името на класа
- Удобно за често използвани константи и методи
- Не прекалявайте – губи се яснотата откъде идва името

```
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;
import static java.lang.Math.pow;

public class Circle {
    private final double r;

    public Circle(double r) { this.r = r; }

    public double area() {
        return PI * pow(r, 2);
    }

    public static double radiusOf(double area) {
        return sqrt(area / PI);
    }
}
```

null и проверки за null

- `null` – референцията не сочи обект
- Извикване на метод върху `null` → `NullPointerException`
- Проверявайте входа рано: `Objects.requireNonNull`
- Съобщението на грешката посочва коя променлива е `null`

```
public class Order {
    private final String customer;

    public Order(String customer) {
        this.customer = Objects.requireNonNull(
            customer, "customer e null");
    }
}

String s = null;
if (s != null && !s.isEmpty()) {
    System.out.println(s.length());
}
String name = Objects.requireNonNullElse(s, "-");
```

Пакети и import

- **Пакет** – група от свързани класове; пространство от имена
- Първи ред във файла: `package ...;`
- Папките съответстват на пакета
- `import` – за да използваме кратко име
- `java.lang` се импортира автоматично

```java
// Файл: src/bg/tu/pik3/bank/Account.java
package bg.tu.pik3.bank;

import java.time.LocalDate;
import java.util.List;
import java.util.ArrayList;

public class Account {
    private final LocalDate opened =
            LocalDate.now();
    private final List<String> log =
            new ArrayList<>();
}
```

06

Записи (record)

Кратки неизменяеми класове за данни (Java 16+)

Клас за данни: преди и сега

Обикновен клас

- `private final` полета
- конструктор
- get методи
- `equals`, `hashCode`, `toString`
- ≈ 40 реда шаблонен код

`record`

- един ред декларация
- всичко отляво се генерира автоматично
- неизменяем по подразбиране
- ясно намерение: „това са данни“

Деклариране на record

- Компонентите се изброяват в скобите
- Генерират се: полета, конструктор, методи за достъп **без** `get`, `equals`, `hashCode`, `toString`
- Записът е `final` – не може да се наследява

```java
public record Point(double x, double y) { }

Point p = new Point(3, 4);
System.out.println(p.x());     // 3.0
System.out.println(p);         // Point[x=3.0, y=4.0]

Point q = new Point(3, 4);
System.out.println(p.equals(q));  // true
System.out.println(p == q);       // false
```

Компактен конструктор и методи

- **Компактният** конструктор проверява данните, без да повтаря параметрите
- Записът може да има методи и статични членове
- Може да имплементира интерфейси

```
public record Student(String name, int fn,
                      double grade) {

    public Student {                    // компактен
        if (grade < 2 || grade > 6) {
            throw new IllegalArgumentException(
                "Оценка: " + grade);
        }
        name = name.strip();
    }

    public boolean excellent() {
        return grade >= 5.5;
    }
}
```

Кога record, кога клас?

Използвайте `record`	Използвайте клас
обекти, които само носят данни	обекти с променящо се състояние
резултати от методи, DTO, ключове в `Map`	обекти с жизнен цикъл (сметка, връзка към сървър)
стойности като `Point`, `Money`, `Range`	когато е нужно наследяване
данни от JSON, CSV, БД	когато трябва да скрием вътрешното представяне

Записи в действие

```
record Grade(String subject, int value) { }

record Student(String name, List<Grade> grades) {

    double average() {
        double sum = 0;
        for (Grade g : grades) sum += g.value();
        return grades.isEmpty() ? 0 : sum / grades.size();
    }
}

var s = new Student("Мария", List.of(
        new Grade("Java", 6),
        new Grade("Бази данни", 5)));

System.out.println(s.name() + ": " + s.average());   // Мария: 5.5
System.out.println(s.grades().get(0));  // Grade[subject=Java, value=6]
```

Записите са удобни за моделиране на данни, които идват от файлове, бази данни или JSON

Добри практики при проектиране на класове

- Един клас – **една отговорност**
- Минимален публичен интерфейс; всичко останало – `private`
- Проверявайте аргументите в конструктора – обектът да е коректен от самото начало
- Предпочитайте неизменяемост и `final` полета
- Описателни имена: съществително за клас, глагол за метод
- Пишете `toString()` – улеснява отстраняването на грешки
- Избягвайте „божествени“ класове с хиляди редове

Кохезия и свързаност

Висока кохезия (добре)

- всички членове на класа служат на **една** цел
- `Fraction` – само операции с дроби
- лесен за разбиране и тестване
- промените са локални

Слаба свързаност (добре)

- класовете знаят **малко** един за друг
- зависят от интерфейси, не от детайли
- смяна на един клас не чупи другите
- противоположното – „всичко зависи от всичко“

Обобщение

- Класът е **шаблон**, обектът – негова **инстанция**, създадена с `new`
- Полетата – `private`; достъпът – чрез методи (**капсулиране**)
- `this` – текущият обект; `this(...)` – друг конструктор
- Ред на инициализация: по подразбиране → инициализатори и блокове → конструктор
- Всички параметри се предават **по стойност** (и референциите)
- `static` – членове на класа; `static final` – константи
- Предпочитаме **неизменяеми** обекти; за данни – `record`

Въпроси за самопроверка

- Каква е разликата между клас и обект?
- Кога компилаторът създава конструктор по подразбиране?
- Защо полетата обикновено са `private`?
- Може ли метод да промени обект, подаден като параметър? А да го подмени?
- Защо статичен метод не може да използва `this`?
- Какво генерира автоматично компилаторът за `record Point(int x, int y)`?
- Какви са предимствата на неизменяемите обекти?

Въпроси?

Следваща лекция: Наследяване и полиморфизъм

